

UANL

FACULTAD DE CIENCIAS

FISICO MATEMATICAS



FCFM

LABORATORIO DE PROGRAMACION ORIENTADA A OBJETOS

PRACTICA #12

Agenda de contactos

2096042

Jimenez Mendieta Angel Gael

INTRODUCCION

En la práctica de la semana pasada logramos levantar nuestra primera aplicación web, pero tenía un detalle importante: los datos solo existían de forma temporal en la memoria. Si apagábamos el servidor, toda la información se perdía. Como en el mundo real los sistemas necesitan guardar los registros de forma permanente, en este laboratorio dimos el siguiente paso al implementar la persistencia de datos. El objetivo principal de esta práctica fue conectar nuestro proyecto de Spring Boot a una base de datos. De esta manera, pudimos capturar información real a través de un formulario web, guardarla y mostrarla dinámicamente en una tabla, logrando un sistema mucho más completo y cercano a lo que se usa allá afuera.

EXPLICACION DEL TEMA

Para lograr que nuestra página realmente guarde la información de los contactos, integramos nuevas herramientas a nuestro modelo MVC que nos automatizan mucho el trabajo con la base de datos:

- **Base de datos H2:** En lugar de instalar y configurar gestores pesados externos, utilizamos esta base de datos embebida. Básicamente, corre internamente junto con nuestra aplicación Java, lo cual la hace perfecta para hacer pruebas rápidas en etapa de desarrollo.
- **JPA (Java Persistence API):** Lo más interesante de esta herramienta es que nos evita tener que escribir código SQL manualmente. Simplemente usando anotaciones (como `@Entity` en nuestra clase `Contacto`), JPA se encarga por detrás de mapear nuestra clase de Java para convertirla en una tabla, y vuelve nuestros atributos las columnas de esa tabla.
- **El Repositorio (JpaRepository):** Esta interfaz funciona como nuestro puente de conexión. Al heredar de ella, Spring nos regala métodos que ya están programados, como `save()`, `findAll()` o `deleteById()`, por lo que podemos hacer operaciones en la base de datos con una sola línea de código.
- **Manejo de Formularios:** En nuestro controlador tuvimos que separar cómo recibimos las peticiones. Usamos `@GetMapping` cuando solo queremos mostrarle una vista al usuario (como la lista o el formulario en blanco), y usamos `@PostMapping` para atrapar la información cuando el usuario le da al botón de guardar. Además, la anotación `@ModelAttribute` nos hizo el favor de empaquetar todo lo que el usuario escribió en el HTML directamente en un objeto de Java, dejándolo listo para guardarse.

CODIGO FUENTE

nuevo.html | contactos.html | ContactoRepository.java | ContactoController.java

agenda-web > src > main > java > com > gremio > agenda_web > controller > ContactoController.java > Language Support for Ja

```java
package com.gremio.agenda_web.controller;

import com.gremio.agenda_web.model.Contacto;
import com.gremio.agenda_web.repository.ContactoRepository;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ContactoController {
    private final ContactoRepository repo;

    public ContactoController(ContactoRepository repo) {
        this.repo = repo;
    }

    @GetMapping("/contactos")
    public String listar(Model model) {
        model.addAttribute("contactos", repo.findAll());
        return "contactos";
    }

    @GetMapping("/nuevo")
    public String formulario(Model model) {
        model.addAttribute("contacto", new Contacto());
        return "nuevo";
    }

    @PostMapping("/guardar")
    public String guardar(@ModelAttribute Contacto contacto) {
        repo.save(contacto);
        return "redirect:/contactos";
    }

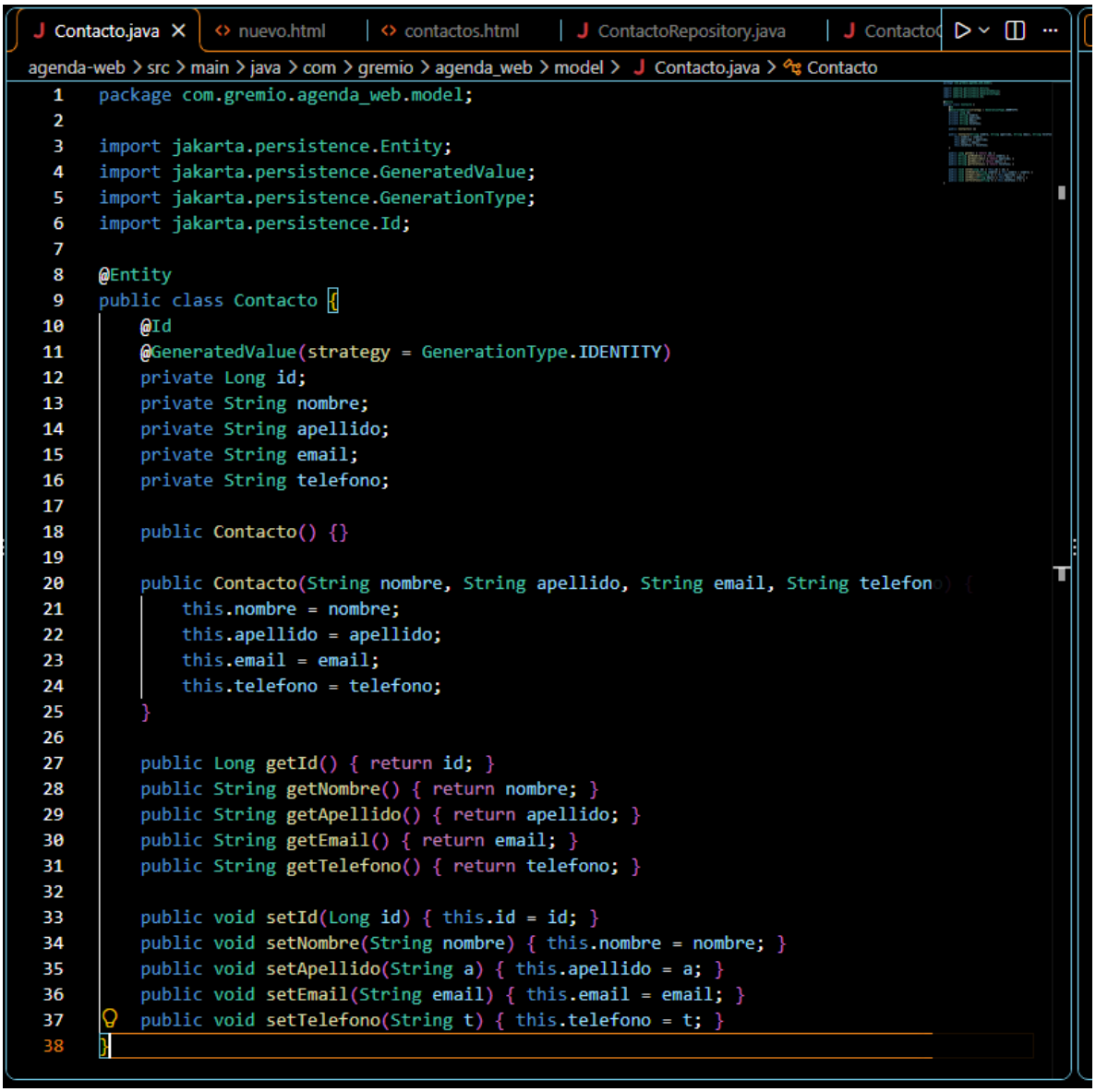
    @GetMapping("/editar/{id}")
    public String editar(@PathVariable Long id, Model model) {
```

gremio > agenda_web > repository > ContactoRepository.java > Language Support for Java(TM) by Red Hat > ContactoRepository

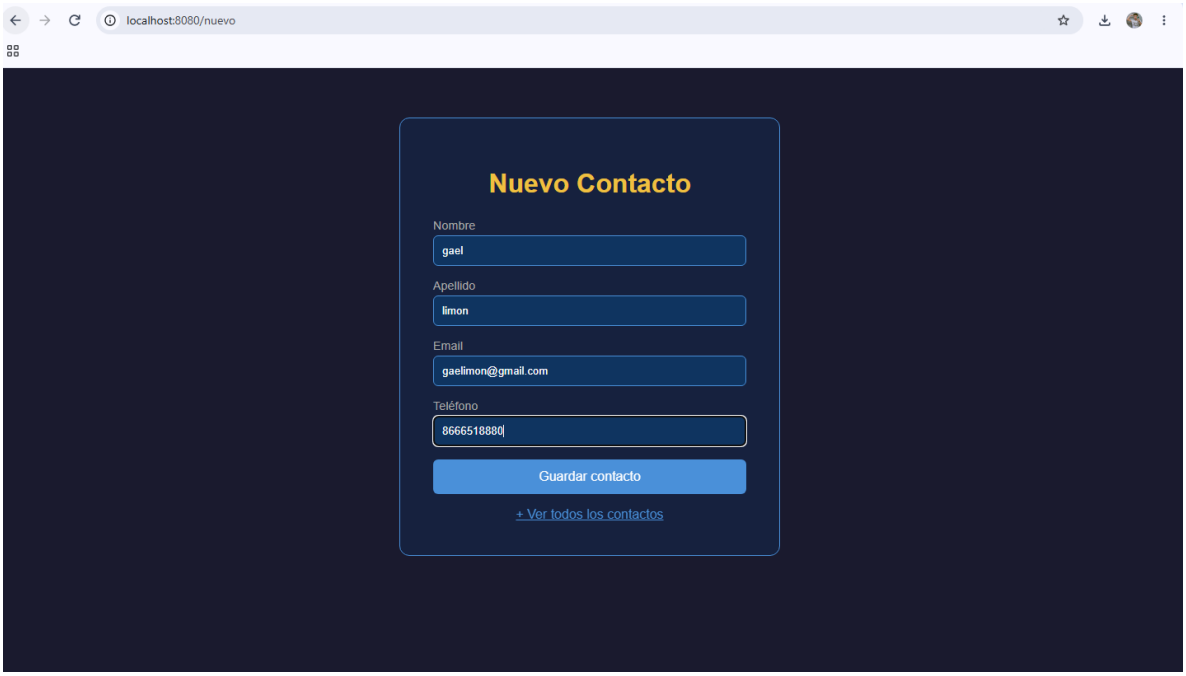
```java
package com.gremio.agenda_web.repository;

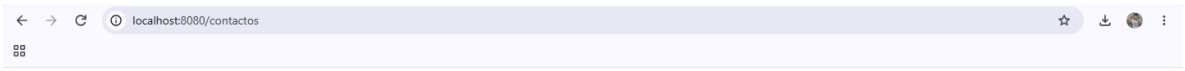
import com.gremio.agenda_web.model.Contacto;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ContactoRepository extends JpaRepository<Contacto, Long> {
}
```


Contacto.java
nuevo.html
contactos.html
ContactoRepository.java
agenda-web > src > main > java > com > gremio > agenda_web > model > Contacto.java > Contacto
package com.gremio.agenda_web.model;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
@Entity
public class Contacto {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String nombre;
private String apellido;
private String email;
private String telefono;
public Contacto() {}
public Contacto(String nombre, String apellido, String email, String telefono) {
this.nombre = nombre;
this.apellido = apellido;
this.email = email;
this.telefono = telefono;
}
public Long getId() { return id; }
public String getNombre() { return nombre; }
public String getApellido() { return apellido; }
public String getEmail() { return email; }
public String getTelefono() { return telefono; }
public void setId(Long id) { this.id = id; }
public void setNombre(String nombre) { this.nombre = nombre; }
public void setApellido(String a) { this.apellido = a; }
public void setEmail(String email) { this.email = email; }
public void setTelefono(String t) { this.telefono = t; }
}

EJECUCION

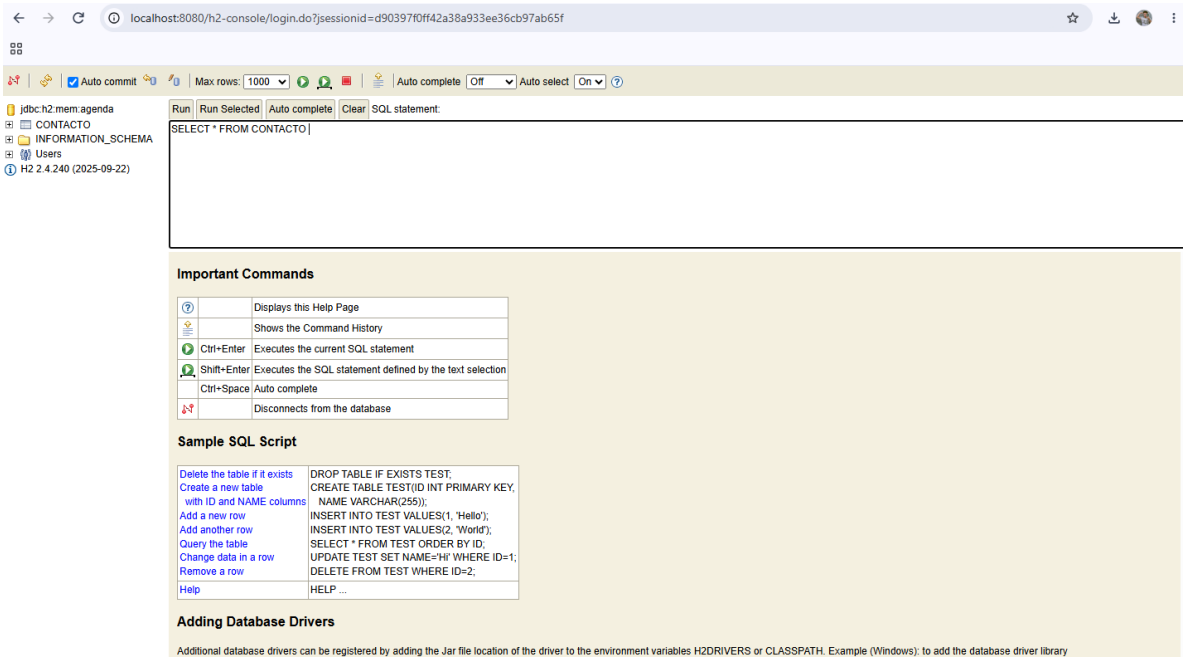




Agenda de Contactos

[+ Nuevo contacto]

Nombre	Apellido	Email	Teléfono	Acciones
gael	limon	gaelimon@gmail.com	8666518880	Editar | Eliminar


localhost:8080/h2-console/login.do?jsessionid=d90397f0ff42a38a933ee36cb97ab65f
Auto commit
Max rows: 1000
Auto complete Off
Auto select On
jdbc:h2:mem:agenda
CONTACTO
INFORMATION_SCHEMA
Users
H2 2.4.240 (2025-09-22)
Run
Run Selected
Auto complete
Clear
SQL statement:
SELECT * FROM CONTACTO
Important Commands
Displays this Help Page
Shows the Command History
Ctrl+Enter
Executes the current SQL statement
Shift+Enter
Executes the SQL statement defined by the text selection
Ctrl+Space
Auto complete
Disconnects from the database
Sample SQL Script
Delete the table if it exists
DROP TABLE IF EXISTS TEST;
Create a new table with ID and NAME columns
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255));
Add a new row
INSERT INTO TEST VALUES(1, 'Hello');
Add another row
INSERT INTO TEST VALUES(2, 'World');
Query the table
SELECT * FROM TEST ORDER BY ID;
Change data in a row
UPDATE TEST SET NAME='Hi' WHERE ID=1;
Remove a row
DELETE FROM TEST WHERE ID=2;
Help
HELP ...
Adding Database Drivers
Additional database drivers can be registered by adding the Jar file location of the driver to the environment variables H2DRIVERS or CLASSPATH. Example (Windows): to add the database driver library

PREGUNTAS

1. ¿Qué hace @Entity? ¿Por qué se requiere el constructor vacío?

   La anotación @Entity indica que la clase de Java se mapeará automáticamente como una tabla en la base de datos. El constructor vacío es un requisito estricto de JPA para poder instanciar internamente los objetos al momento de recuperar registros de la base de datos.

2. ¿Por qué ContactoRepository no tiene ningún método escrito pero aun así puedes llamar findAll() y save()?

   Debido a que la interfaz extiende de JpaRepository, hereda automáticamente la implementación de los métodos estándar de acceso a datos (como save() o findAll()) sin necesidad de que el desarrollador escriba manualmente las sentencias SQL.

3. ¿Qué diferencia hay entre @GetMapping y @PostMapping? ¿Cuándo usarías cada uno?

   La anotación @GetMapping se emplea para interceptar peticiones de visualización de páginas o datos (como acceder a la lista o al formulario vacío), mientras que @PostMapping intercepta exclusivamente los datos que el usuario envía a través del formulario HTML para procesarlos.

4. Agregar modulos de delete y update

   Para implementar las operaciones de eliminar y actualizar, se añadieron los controladores /editar/{id} y /eliminar/{id}. El método de edición recupera el contacto con findById() y reutiliza la vista del formulario, mientras que la eliminación emplea deleteById() y redirige la vista a la lista principal.